

Web Scraper Lambda

A Flask-based AWS Lambda that scrapes HTML tables from websites and provides a filtering API for the collected data.

Features

- Scheduled web scraping via CloudWatch Events
- Data storage in S3 as JSON
- REST API for data filtering
- Automated deployment script

Prerequisites

- Python 3.9+
- AWS CLI configured with appropriate permissions
- AWS IAM role for Lambda with S3 and CloudWatch permissions
- Bash-compatible shell

Quick Start

1. Clone this repository
2. Edit configuration in `deploy.sh`
3. Make the deployment script executable: `chmod +x deploy.sh`
4. Run the deployment: `./deploy.sh`

Configuration

Edit the following variables in `deploy.sh`:

bash

```
LAMBDA_NAME="web-scraper-lambda"           # Name of your Lambda function
REGION="us-east-1"                         # Your AWS region
ROLE_ARN="arn:aws:iam::123456789012:role/..." # Your Lambda execution role ARN
S3_BUCKET_NAME="your-bucket-name"          # S3 bucket for deployment and data storage
SCHEDULE_EXPRESSION="cron(0 0 * * ? *)"     # CloudWatch schedule expression
SCHEDULE_RULE_NAME="daily-web-scraper-rule" # Name of the CloudWatch rule
```

Environment variables in the Lambda:

S3_BUCKET_NAME – S3 bucket to store scraped data
S3_JSON_KEY – Path/filename in S3 for the JSON data
TARGET_URL – Website URL to scrape

Local Testing

1. Create a virtual environment:

```
python -m venv venv  
source venv/bin/activate # On Windows: venv\Scripts\activate
```

2. Install dependencies:

```
pip install -r requirements.txt
```

3. For testing the Flask app locally:

```
python app.py
```

4. For testing the Lambda handler function:

```
python  
  
from app import scheduled_scraper  
result = scheduled_scraper({}, {})  
print(result)
```

API Endpoints

GET /scrape (Optional)

Manually triggers the scraping process.

Response:

```
json  
  
{  
  "success": true,  
  "message": "Data scraped and saved to S3",  
  "record_count": 42  
}
```

POST /data

Retrieves and filters the scraped data.

Request Body:

json

```
{  
  "field1": "value1",  
  "field2": "value2"  
}
```

Response:

json

```
{  
  "data": [  
    {  
      "field1": "value1",  
      "field2": "value2",  
      "field3": "value3"  
    }  
  ],  
  "count": 1  
}
```

Deployment

The included `deploy.sh` script handles:

1. Installing dependencies
2. Creating a deployment package
3. Creating/updating the Lambda function
4. Setting up CloudWatch Events for scheduled execution

To deploy:

```
./deploy.sh
```

Customizing the Scraper

Modify the `scrape_and_save_data` function in `app.py` to adjust the scraping logic. The default implementation looks for an HTML table element and extracts rows and columns.

Update the CSS selector to target the specific table on your target website:

python

```
# Find the table - modify selector based on actual HTML structure
table = soup.select_one('table.your-table-class') # Adjust selector as needed
```

Troubleshooting

- **Lambda Timeout:** If scraping takes too long, increase the Lambda timeout in `deploy.sh`
- **Missing Permissions:** Ensure the Lambda execution role has permissions for S3 and CloudWatch
- **Scraping Issues:** Check the CloudWatch Logs for the Lambda to see detailed error messages

Required IAM Permissions

The Lambda execution role needs:

- `s3:PutObject` and `s3:GetObject` for the target S3 bucket
- `logs:CreateLogGroup`, `logs:CreateLogStream`, and `logs:PutLogEvents`
- `lambda:InvokeFunction` permission for CloudWatch Events

License

MIT